In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# setting display options so dataframes are readable
pd.set_option('display.max_columns', 50)
pd.set_option('display.float_format', '{:.2f}'.format)
sns.set_theme(style='whitegrid')
print("imports done")
```

```
imports done
```

In [2]:
```
import os
from google.colab import drive

drive.mount('/content/drive', force_remount=False)

# setting the base project path
base_path = '/content/drive/MyDrive/ML/stock-lens/'

# creating data and notebooks folders if they don't already exist
os.makedirs(base_path + 'data/',      exist_ok=True)
os.makedirs(base_path + 'notebooks/', exist_ok=True)

data_path = base_path + 'data/'

print("project folder ready")
print("data path:     ", data_path)
print("notebooks path:", base_path + 'notebooks/')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
project folder ready
data path:      /content/drive/MyDrive/ML/stock-lens/data/
notebooks path: /content/drive/MyDrive/ML/stock-lens/notebooks/
```

In [3]:
```
kaggle_files = ['prices-split-adjusted.csv', 'fundamentals.csv', 'securities
files_exist = all(os.path.exists(data_path + f) for f in kaggle_files)

if not files_exist:
    os.environ['KAGGLE_USERNAME'] = 'your_kaggle_username'
    os.environ['KAGGLE_KEY']      = 'your_api_key'
    os.system('pip install kaggle -q')
    os.system(f'kaggle datasets download -d dgawlik/nyse -p {data_path} --un
    print("download complete")
else:
    print("files already present — skipping download")

# loading all three datasets
prices       = pd.read_csv(data_path + 'prices-split-adjusted.csv', parse_da
fundamentals = pd.read_csv(data_path + 'fundamentals.csv')
securities   = pd.read_csv(data_path + 'securities.csv')

print("\nprices shape:      ", prices.shape)
print("fundamentals shape:", fundamentals.shape)
```

```
print("securities shape:  ", securities.shape)
print("\ndate range:", prices['date'].min(), "to", prices['date'].max())
print("unique tickers:", prices['symbol'].nunique())
```

```
files already present — skipping download

prices shape:       (851264, 7)
fundamentals shape: (1781, 79)
securities shape:   (505, 8)

date range: 2010-01-04 00:00:00 to 2016-12-30 00:00:00
unique tickers: 501
```

In [4]:
```
# loading the annual financial metrics from SEC 10-K filings
fundamentals = pd.read_csv(data_path + 'fundamentals.csv')

print("shape:", fundamentals.shape)
print("\ncolumns:", fundamentals.columns.tolist())
fundamentals.head()
```

```
shape: (1781, 79)

columns: ['Unnamed: 0', 'Ticker Symbol', 'Period Ending', 'Accounts Payable',
'Accounts Receivable', "Add'l income/expense items", 'After Tax ROE', 'Capita
l Expenditures', 'Capital Surplus', 'Cash Ratio', 'Cash and Cash Equivalent
s', 'Changes in Inventories', 'Common Stocks', 'Cost of Revenue', 'Current Ra
tio', 'Deferred Asset Charges', 'Deferred Liability Charges', 'Depreciation',
'Earnings Before Interest and Tax', 'Earnings Before Tax', 'Effect of Exchang
e Rate', 'Equity Earnings/Loss Unconsolidated Subsidiary', 'Fixed Assets', 'G
oodwill', 'Gross Margin', 'Gross Profit', 'Income Tax', 'Intangible Assets',
'Interest Expense', 'Inventory', 'Investments', 'Liabilities', 'Long-Term Deb
t', 'Long-Term Investments', 'Minority Interest', 'Misc. Stocks', 'Net Borrow
ings', 'Net Cash Flow', 'Net Cash Flow-Operating', 'Net Cash Flows-Financin
g', 'Net Cash Flows-Investing', 'Net Income', 'Net Income Adjustments', 'Net
Income Applicable to Common Shareholders', 'Net Income-Cont. Operations', 'Ne
t Receivables', 'Non-Recurring Items', 'Operating Income', 'Operating Margi
n', 'Other Assets', 'Other Current Assets', 'Other Current Liabilities', 'Oth
er Equity', 'Other Financing Activities', 'Other Investing Activities', 'Othe
r Liabilities', 'Other Operating Activities', 'Other Operating Items', 'Pre-T
ax Margin', 'Pre-Tax ROE', 'Profit Margin', 'Quick Ratio', 'Research and Deve
lopment', 'Retained Earnings', 'Sale and Purchase of Stock', 'Sales, General
and Admin.', 'Short-Term Debt / Current Portion of Long-Term Debt', 'Short-Te
rm Investments', 'Total Assets', 'Total Current Assets', 'Total Current Liabi
lities', 'Total Equity', 'Total Liabilities', 'Total Liabilities & Equity',
'Total Revenue', 'Treasury Stock', 'For Year', 'Earnings Per Share', 'Estimat
ed Shares Outstanding']
```

Out[4]:

	Unnamed: 0	Ticker Symbol	Period Ending	Accounts Payable	Accounts Receivable	Add'l income/expense items	Af
0	0	AAL	2012-12-31	3068000000.00	-222000000.00	-1961000000.00	23
1	1	AAL	2013-12-31	4975000000.00	-93000000.00	-2723000000.00	67
2	2	AAL	2014-12-31	4668000000.00	-160000000.00	-150000000.00	143
3	3	AAL	2015-12-31	5102000000.00	352000000.00	-708000000.00	135
4	4	AAP	2012-12-29	2409453000.00	-89482000.00	600000.00	32

5 rows × 79 columns

In [5]:
```
# loading sector and industry info per ticker
securities = pd.read_csv(data_path + 'securities.csv')

print("shape:", securities.shape)
print("\ncolumns:", securities.columns.tolist())
securities.head()
```

```
shape: (505, 8)

columns: ['Ticker symbol', 'Security', 'SEC filings', 'GICS Sector', 'GICS Su
b Industry', 'Address of Headquarters', 'Date first added', 'CIK']
```

Out[5]:

	Ticker symbol	Security	SEC filings	GICS Sector	GICS Sub Industry	Address of Headquarters
0	MMM	3M Company	reports	Industrials	Industrial Conglomerates	St. Paul, Minnesota
1	ABT	Abbott Laboratories	reports	Health Care	Health Care Equipment	North Chicago, Illinois
2	ABBV	AbbVie	reports	Health Care	Pharmaceuticals	North Chicago, Illinois
3	ACN	Accenture plc	reports	Information Technology	IT Consulting & Other Services	Dublin, Ireland
4	ATVI	Activision Blizzard	reports	Information Technology	Home Entertainment Software	Santa Monica, California

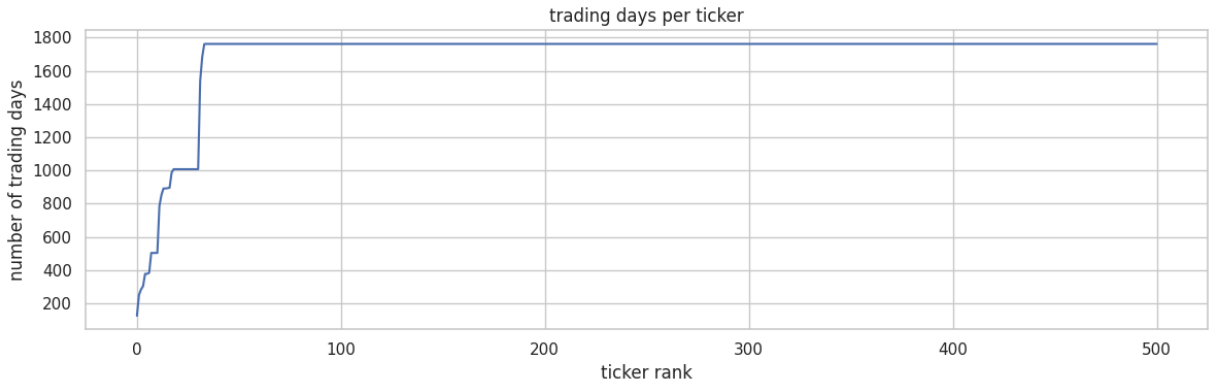
In [6]:
```
# checking how many nulls exist in each column
null_counts = prices.isnull().sum()
print("null counts in prices:\n", null_counts)
```

```
null counts in prices:
 date      0
symbol    0
open      0
close     0
low       0
high      0
volume    0
dtype: int64
```

In [7]:
```
print("null counts in fundamentals:\n", fundamentals.isnull().sum())
print("\nnull counts in securities:\n", securities.isnull().sum())
```

```
null counts in fundamentals:
 Unnamed: 0                          0
Ticker Symbol                        0
Period Ending                        0
Accounts Payable                     0
Accounts Receivable                  0
                                   ...
Total Revenue                        0
Treasury Stock                       0
For Year                           173
Earnings Per Share                 219
Estimated Shares Outstanding       219
Length: 79, dtype: int64

null counts in securities:
 Ticker symbol                0
Security                      0
SEC filings                   0
GICS Sector                   0
GICS Sub Industry             0
Address of Headquarters       0
Date first added            198
CIK                           0
dtype: int64
```

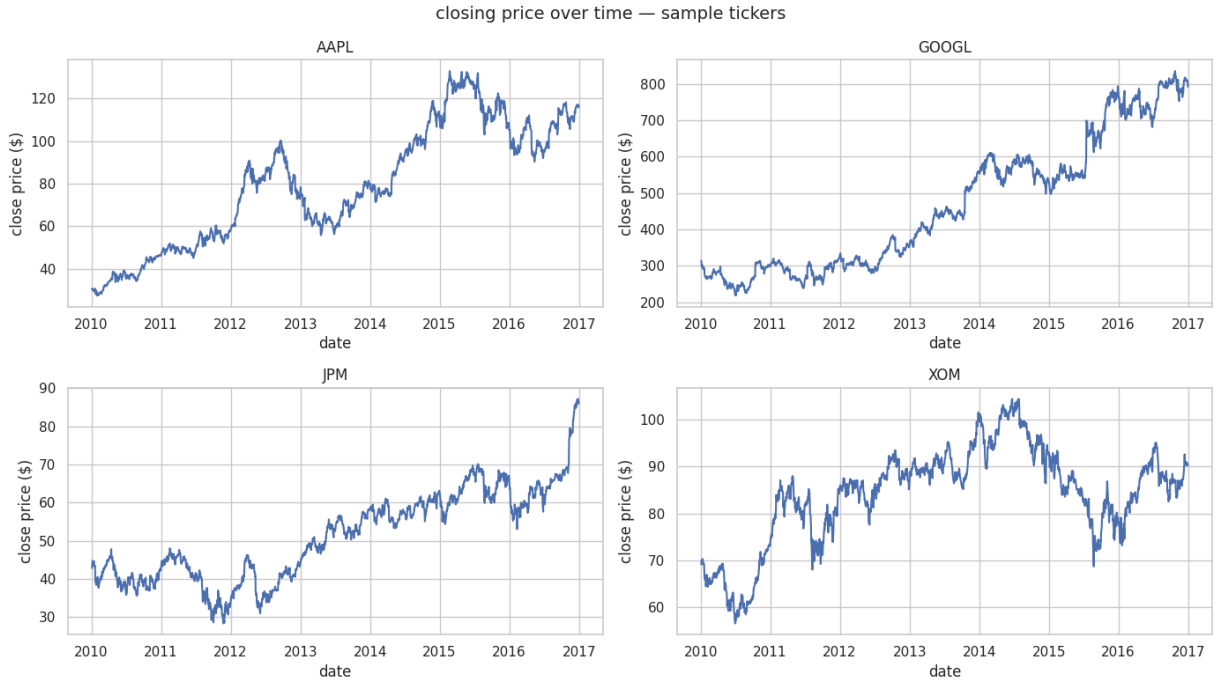
In [8]:
```
# checking how many trading days each ticker has — should be roughly equal
days_per_ticker = prices.groupby('symbol')['date'].count().sort_values()

plt.figure(figsize=(12, 4))
plt.plot(days_per_ticker.values)
plt.title('trading days per ticker')
plt.xlabel('ticker rank')
plt.ylabel('number of trading days')
plt.tight_layout()
plt.show()

print("min days:", days_per_ticker.min(), "| max days:", days_per_ticker.max
print("tickers with fewer than 1000 days:", (days_per_ticker < 1000).sum())
```




```
min days: 126 | max days: 1762
tickers with fewer than 1000 days: 18
```

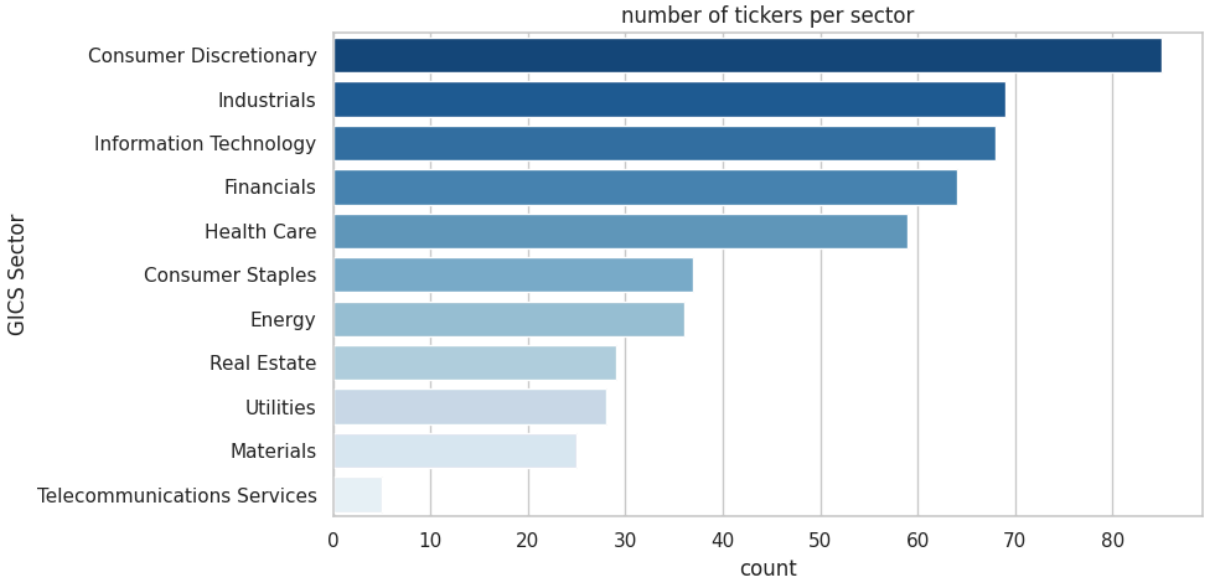
In [9]:
```python
# looking at the overall distribution of closing prices across all tickers
plt.figure(figsize=(10, 4))
sns.histplot(prices['close'], bins=100, kde=True)
plt.title('distribution of closing prices (all tickers, all dates)')
plt.xlabel('closing price ($)')
plt.tight_layout()
plt.show()
```

In [10]:
```python
# plotting closing price over time for 4 well-known tickers to sanity check
example_tickers = ['AAPL', 'GOOGL', 'JPM', 'XOM']

fig, axes = plt.subplots(2, 2, figsize=(14, 8))
axes = axes.flatten()

for i, ticker in enumerate(example_tickers):
    ticker_data = prices[prices['symbol'] == ticker]
    axes[i].plot(ticker_data['date'], ticker_data['close'])
    axes[i].set_title(ticker)
    axes[i].set_xlabel('date')
    axes[i].set_ylabel('close price ($)')

plt.suptitle('closing price over time — sample tickers', fontsize=14)
plt.tight_layout()
plt.show()
```



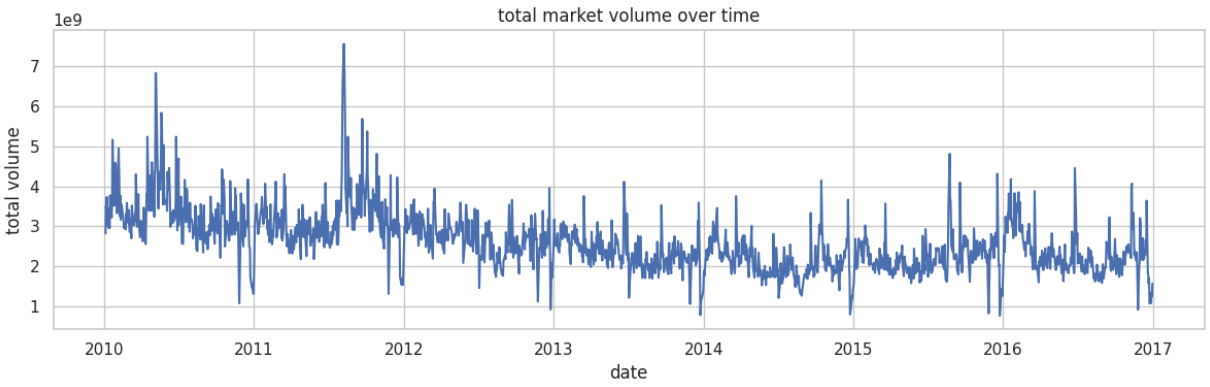
In [11]:
```
# checking how many tickers fall into each sector
sector_counts = securities['GICS Sector'].value_counts()

plt.figure(figsize=(10, 5))
sns.barplot(x=sector_counts.values, y=sector_counts.index, palette='Blues_r'
plt.title('number of tickers per sector')
plt.xlabel('count')
plt.tight_layout()
plt.show()

print(sector_counts)
```

```
/tmp/ipykernel_14201/2958505435.py:5: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed i
n v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the sa
me effect.

  sns.barplot(x=sector_counts.values, y=sector_counts.index, palette='Blues_
r')
```




```
GICS Sector
Consumer Discretionary          85
Industrials                     69
Information Technology          68
Financials                      64
Health Care                     59
Consumer Staples                37
Energy                          36
Real Estate                     29
Utilities                       28
Materials                       25
Telecommunications Services      5
Name: count, dtype: int64
```

In [12]:
```
# checking total market volume over time to spot any anomalies
daily_volume = prices.groupby('date')['volume'].sum()

plt.figure(figsize=(12, 4))
plt.plot(daily_volume.index, daily_volume.values)
plt.title('total market volume over time')
plt.xlabel('date')
plt.ylabel('total volume')
plt.tight_layout()
plt.show()
```



In [13]:
```
# printing a clean summary of all three datasets before moving to feature en
print("=== DATASET SUMMARY ===")
print(f"prices      : {prices.shape[0]:,} rows | {prices['symbol'].nunique()
print(f"fundamentals: {fundamentals.shape[0]:,} rows | {fundamentals.shape[1
print(f"securities  : {securities.shape[0]:,} rows | {securities['GICS Secto
```

```
=== DATASET SUMMARY ===
prices      : 851,264 rows | 501 tickers | 2010-01-04 to 2016-12-30
fundamentals: 1,781 rows | 79 columns
securities  : 505 rows | 11 sectors
```

In [ ]:
```
import os
import json
from google.colab import drive, _message, userdata

# pulling credentials from colab secrets
github_token = userdata.get('GITHUB_TOKEN')
github_email = userdata.get('GITHUB_EMAIL')
github_name  = userdata.get('GITHUB_NAME')   # add this secret too — e.g. Si

# saving the notebook to google drive
notebook_name = '01_data_loading'  # change this per notebook — no extension
notebook_path = f'/content/drive/MyDrive/ML/stock-lens/notebooks/{notebook_n

notebook_json = _message.blocking_request('get_ipynb', request='', timeout_s
with open(notebook_path, 'w') as f:
    json.dump(notebook_json['ipynb'], f)
print("saved to drive ")

# installing pdf conversion dependencies
os.system('apt-get install -y libatk1.0-0 libatk-bridge2.0-0 libcups2 libxkb
os.system('pip install -q nbconvert[webpdf] playwright')
os.system('playwright install chromium')

# converting notebook to pdf
os.system(f'jupyter nbconvert --to webpdf --allow-chromium-download "{notebo
pdf_path = notebook_path.replace('.ipynb', '.pdf')
print("pdf created!!" if os.path.exists(pdf_path) else "pdf creation failed"

# cloning repo if not already present
repo_path = '/content/stock-lens'
if not os.path.exists(repo_path):
    os.system(f'git clone https://github.com/SidRoy97/stock-lens.git {repo_p

# creating notebooks folder if missing
os.makedirs(f'{repo_path}/notebooks', exist_ok=True)

# copying notebook and pdf into repo
os.system(f'cp "{notebook_path}" "{repo_path}/notebooks/{notebook_name}.ipyn
if os.path.exists(pdf_path):
    os.system(f'cp "{pdf_path}" "{repo_path}/notebooks/{notebook_name}.pdf"'

# configuring git with user credentials
os.system(f'git -C {repo_path} config user.email "{github_email}"')
os.system(f'git -C {repo_path} config user.name "{github_name}"')
os.system(f'git -C {repo_path} add notebooks/')
```

```
commit = os.popen(f'git -C {repo_path} commit -m "update {notebook_name} not
print("commit:", commit)

# pushing to github
remote = f'https://{github_token}@github.com/SidRoy97/stock-lens.git'
push   = os.popen(f'git -C {repo_path} push {remote} main 2>&1').read()
print("push:", push)
```